Contents

Chapter 1

Data tables in SQL

Most organizations keep their data as tables inside a relational database management system (compared to pandas, CSV, or spreadsheets). Developers talk to those systems using a language called SQL ("Structured Query Language").

Some relational database managers are pricey products you may have heard of before, such as Oracle or Microsoft SQL Server. Some are free, such as PostgreSQL or MySQL. These are server software that client programs talk to over a company's network.[1]

There is a library, called `sqlite`, that lets us create files that hold tables. We can use SQL to create, edit, and browse those tables. sqlite is free, fast, and very easy to install. We will use sqlite instead of a networked database management system.

If you look in your digital resources, you will find a file called `bikes.db`. We created this file using sqlite, and now you will use sqlite to access it.

In the terminal, get to the directory where `bikes.db` lives. To open the sqlite tool on that file:

```
> sqlite3 bikes.db
```

(If your system complains that there is no sqlite3 tool, you need to install sqlite. See this website: https://sqlite.org/)

Please follow along: type each command shown here into the terminal and see what happens.

We mostly run SQL commands in this tool, but there are a few non-SQL commands that all start with a period. To see the tables and their columns, you can run `.schema`:

```
sqlite> .schema
CREATE TABLE bike (bike_id int PRIMARY KEY, brand text, size int,
          purchase_price real, purchase_date date, status text);
```

[1] It should be noted that many notetaking and file storage applications allow you to run SQL-like queries to search your files to meet certain criteria. For example, Obsidian, the Markdown notetaking app, has a plugin called Dataview, which allows you to run searches for notes matching certain *metadata* attribute criteria. See more about this here: https://github.com/blacksmithgu/obsidian-dataview

That is the SQL command that we used to create the `bike` table. You can see all the columns and their types.

You want to see all the rows of data in that table?

```
sqlite> select * from bike;
4997391|GT|57|269.61|2009-05-03|rented
5429447|Cannondale|50|215.91|2002-02-17|broken
5019171|Trek|58|251.17|1985-07-11|rented
3000288|Cannondale|57|211.08|1993-01-05|broken
880965|GT|52|281.75|1995-08-02|available
...
```

You will see 1000 rows of data!

The SQL language is not case-sensitive, so you can also write it like this:

```
sqlite> SELECT * FROM BIKE;
```

Often, you will see SQL with just the SQL keywords in all caps:

```
sqlite> SELECT * FROM bike;
```

The semicolon is not part of SQL, but it tells sqlite that you are done writing a command and that it should be executed.

SQL lets you choose which columns you would like to see. The asterisk (FIXME) used above signifies all columns, and the `bike_id, brand` only gets the bike's id and brand from the dataframe:

```
sqlite> SELECT bike_id, brand FROM bike;
4997391|GT
5429447|Cannondale
5019171|Trek
3000288|Cannondale
...
```

Using WHERE, SQL lets you use conditions to decide which rows you would like to see, and can be combined with the common operators `AND`, `OR`, and `NOT`:

```
sqlite> SELECT * FROM bike WHERE purchase_date > '2009-01-01' AND brand = 'GT';
4997391|GT|57|269.61|2009-05-03|rented
326774|GT|56|165.0|2009-06-27|available
```

```
264933|GT|52|302.43|2009-07-09|available
5931243|GT|55|173.56|2009-11-26|rented
4819848|GT|51|221.71|2009-12-11|rented
9347713|GT|52|232.32|2009-06-13|available
3019205|GT|58|262.94|2009-08-22|available
```

Using DISTINCT, SQL lets you get just one copy of each value:

```
sqlite> SELECT DISTINCT status FROM bike;
rented
broken
available

Busted
Flat tire
good
out
Rented
```

You can also edit these rows. For example, if you wanted every status that is `Busted` to be changed to `broken`, you can use an `UPDATE` statement with a `SET`:

```
sqlite> UPDATE bike SET status='broken' WHERE status='Busted';
sqlite> SELECT DISTINCT status FROM bike;
rented
broken
available
Flat tire
good
out
Rented
```

You can insert new rows:

```
sqlite> INSERT INTO bike (bike_id, brand, size, purchase_price, purchase_date,
↪  status)
  ...> VALUES (1, 'GT', 53, 123.45, '2020-11-13', 'available');
sqlite> SELECT * FROM bike WHERE bike_id = 1;
1|GT|53|123.45|2020-11-13|available
```

Note that the bike_id here must be *unique*.

You can delete rows:

```
sqlite> DELETE FROM bike WHERE bike_id = 1;
```

```
sqlite> SELECT * FROM bike WHERE bike_id = 1;
```

To get out of sqlite, type `.exit`.

Exercise 1 SQL Query

Execute an SQL query that returns the `bike_id` (no other columns) of every Trek bike that cost more than $300.

Working Space

Answer on Page 41

1.1 Using SQL from Python

The people behind sqlite created a library for Python that lets you execute SQL and fetch the results from inside a python program.

Let's create a simple program that fetches and displays the bike ID and purchase date of every Trek bike that cost more than $300.

Create a file called `report.py`:

```
import sqlite3 as db

con = db.connect('bikes.db')
cur = con.cursor()

cur.execute("SELECT bike_id, purchase_date FROM bike WHERE purchase_price > 330 AND brand='Trek'")
rows = cur.fetchall()

today = datetime.date.today()
for row in rows:
  print(f"Bike {row[0]}, purchased {row[1]}")

con.close()
```

When you execute it, you should see:

```
> python3 report.py
```

```
Bike 4128046, purchased 2007-08-06
Bike 7117808, purchased 1995-03-12
Bike 7176903, purchased 1986-07-03
Bike 827899, purchased 2009-03-14
Bike 363983, purchased 1970-08-16
```

Chapter 2

Diodes, Rectifiers, and Photovoltaics

Early in this course, we discussed some basic electronics components: resistors, capacitors, and inductors. In this chapter, we are going to introduce a semiconductor: *Diodes* ensure that current flows in only one direction.

You will learn a lot about semiconductors from studying diodes. We can then apply what you have learned to solar cells, transistors, and integrated circuits like CPUs.

We are going to focus on semiconductors made of silicon. You can make semiconductors out of other materials, but silicon has proved to be very versatile and cost-effective.

2.1 Silicon

A silicon atom has 14 protons and 14 electrons. The inner 10 electrons are arranged in two complete and stable shells. The outer four electrons are in an incomplete shell, which allows them to form bonds with other atoms. We call these *valence electrons* because they aren't tightly bound to their atoms. A complete outer shell would have eight electrons.

Atoms can share electrons to get complete shells. This is called *covalent bonding*. When atoms share electrons, they form a *covalent bond*. The electrons are shared between the atoms, and the atoms are held together by the shared electrons.

In a silicon crystal, each silicon atom shares its outer four electrons with four neighboring atoms. With its four electrons and an electron borrowed from each of it four neighbors, each silicon atom has eight electrons in its outer shell. This is a complete shell, and the atoms are stable. This is why glass is hard and stable. It also why electricity doesn't flow through glass – all the electrons are tightly bound to their atoms.

Phosphorus has five valence electrons. If you put a phosphorus atom into silicon crystal, it shares four of its electrons with its neighbors, but it has one electron left over after the complete shell of eight is built. As you add more phosphorus to a silicon crystal, it conducts more electricity. We call this *doping*.

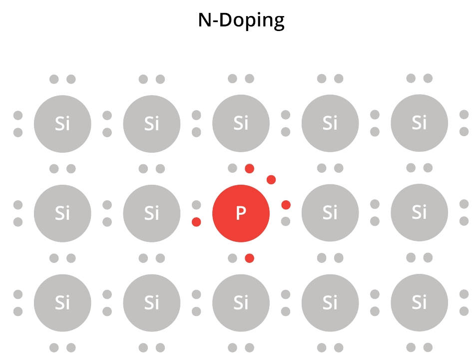


Figure 2.1: Phosphorus atom in a silicon crystal

Because this kind of doping frees up some electrons, which are negatively charged, we call this *n-doping*.

Boron has only three valence electrons. If you put a boron atom into silicon crystal, it shares three of its electrons with its neighbors, which leaves a "hole" that would like to be filled by an electron. We call this *p-doping*.

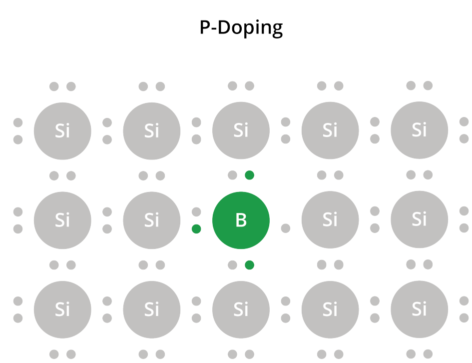


Figure 2.2: Boron atom in a silicon crystal

Both n-doped and p-doped silicon are conductors of electricity. In n-doped silicon, we say the loose electrons are the *carriers* of the current. In p-doped silicon, we say the holes are the carriers.

Note that doped silicon is still mostly silicon. In normal doping, maybe 1 in 10,000 atoms are replaced with boron or phosphorus.

2.2 Diodes

A diode is a semiconductor device that allows current to flow in one direction only.

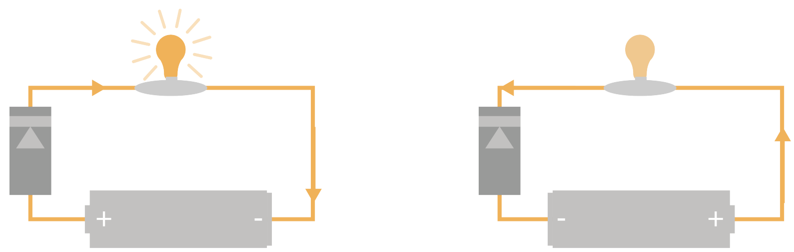


Figure 2.3: Current flows in one direction through the diode, not in the other direction.

Diodes can be made with silicon that has been n-doped and p-doped. A diode looks like this:

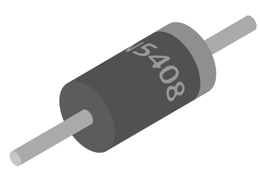


Figure 2.4: What Diodes Look Like

We say that current flows from the positive terminal to the negative terminal of a battery. However, remember that the electrons are actually moving from the negative terminal to the positive terminal. This is a common source of confusion in any discussion of diodes.

One end of the diode is called the *anode*. The anode is p-doped. The other end is called the *cathode*. The cathode is n-doped.

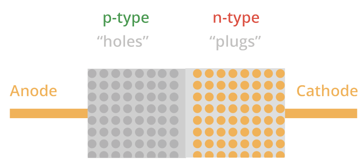


Figure 2.5: A p-n Diode

The region close to where the anode and cathode meet is called the *depletion region*. In this space, the spare electrons in the cathode (the n-doped material) migrate over to fill the holes in anode (the p-doped material). This creates an electric field: there are more electrons than protons on the anode side, and more protons than electrons on the cathode side. (How strong is this field? It is typically about 0.7 volts.)

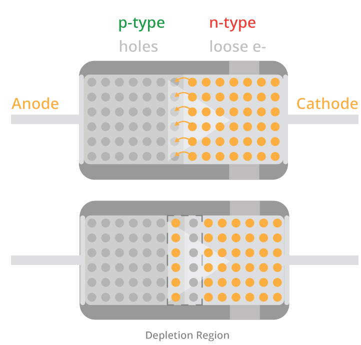


Figure 2.6: The depletion region in a diode

What happens if we overpower that 0.7 volts? What if we put 2 volts in the opposite direction? This shrinks the depletion region and the diode becomes a conductor. For example, if the negative end of a battery is connected to the cathode of a diode and the positive end is connected to the anode, current will flow through the diode.

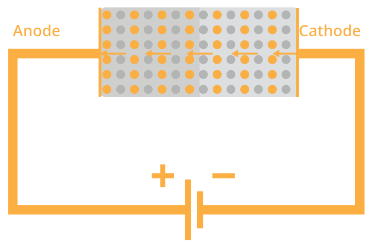


Figure 2.7: Electrons flow through the diode

If we reverse the battery, the depletion region expands, and the diode becomes an insulator.

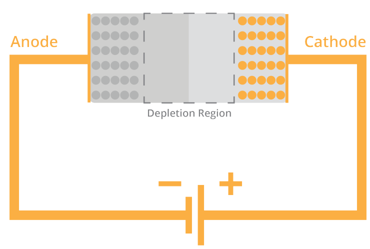


Figure 2.8: Reverse the electric field? No flow.

Here is the symbol for a diode:

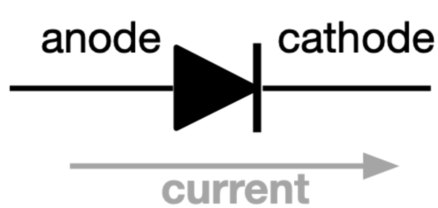


Figure 2.9: Diode Symbol

Current will flow in the direction of the arrow. It will not flow in the opposite direction.

2.2.1 Rectifiers

One of the really useful things we can do with a diode is to convert alternating current (AC) into direct current (DC). This is called rectification. Remember that AC pushes and pulls the electrons back and forth in the wire. Rectification ensures that the electrons only move in one direction.

A *half-wave rectifier* uses a single diode to ensure that the current flows in one direction through the load.

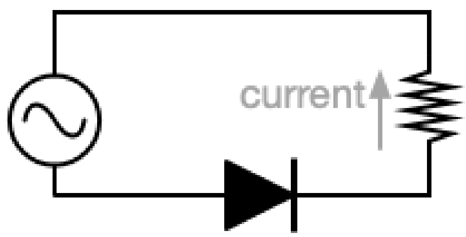


Figure 2.10: Half-wave Rectifier

Current flows in the direction of the arrow on the diode. If the voltage is reversed, the diode will block the current from moving.

A *full-wave rectifier* uses four diodes to swap the direction of the current through the load when necessary.

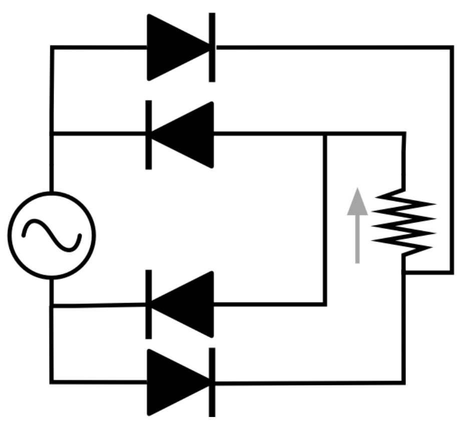


Figure 2.11: Full-wave Rectifier

2.2.2 Photovoltaics

Photovoltaics (or "Solar cells") are similar to diodes. They have a p-n junction with a 0.7 volt electrical field. The difference is that sunlight is allowed to shine on the cathode side. When an atom is struck by a photon of light energy, the electron becomes excited. Sometimes is it excited enough to leap across that 0.7 volt barrier.

When it leaps across, there is an extra electron on the anode side of the barrier and an extra proton on the cathode side. The electron is pushed back to where it came from, but it can't go back over the barrier (unless more than 0.7 volts of charge have built up across the barrier). So, it goes the long way: through the load.

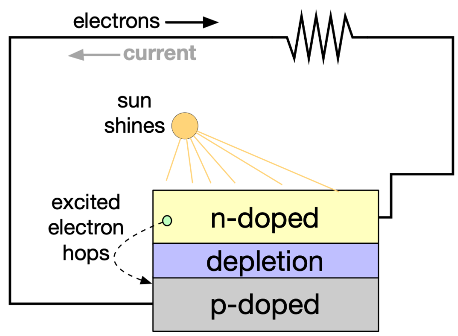


Figure 2.12: Solar Cell

Chapter 3

Trees

Trees are one of the most versatile and widely used data structures in computer science. A tree is a hierarchical data structure consisting of nodes, where each node has a value and a set of references to its child nodes. The node at the top of the hierarchy is called the root, and nodes with the same parent are called siblings.

Chapter 4

Tree Traversal

Chapter 5

Finding Shortest Paths

Chapter 6

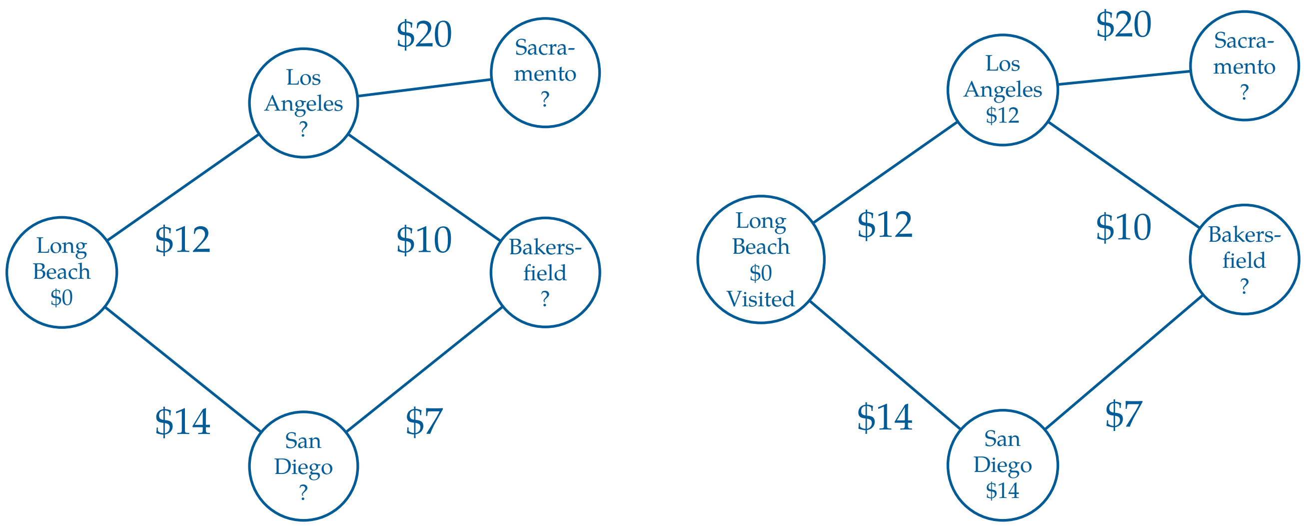
Dijkstra's Algorithm

Edsger W. Dijkstra was a great Dutch computer scientist. He came up with an algorithm for finding the cheapest path through a graph with weighted edges. Today it is known as *Dijkstra's Algorithm*. It is used in a wide variety of common problems, and is also both simple and elegant.

6.1 Algorithm Description

You are going to mark each node with how much it would cost to get there from some chosen origin node. For example, if you are shipping a container from Long Beach, you will mark each city with the cost of getting the container to that city.

You start by marking the price for Long Beach to zero (the container is already there). You then mark each adjacent city with the cost on the edge (figure 6.1a). Next, you declare Long Beach to be "visited" (figure 6.1b).



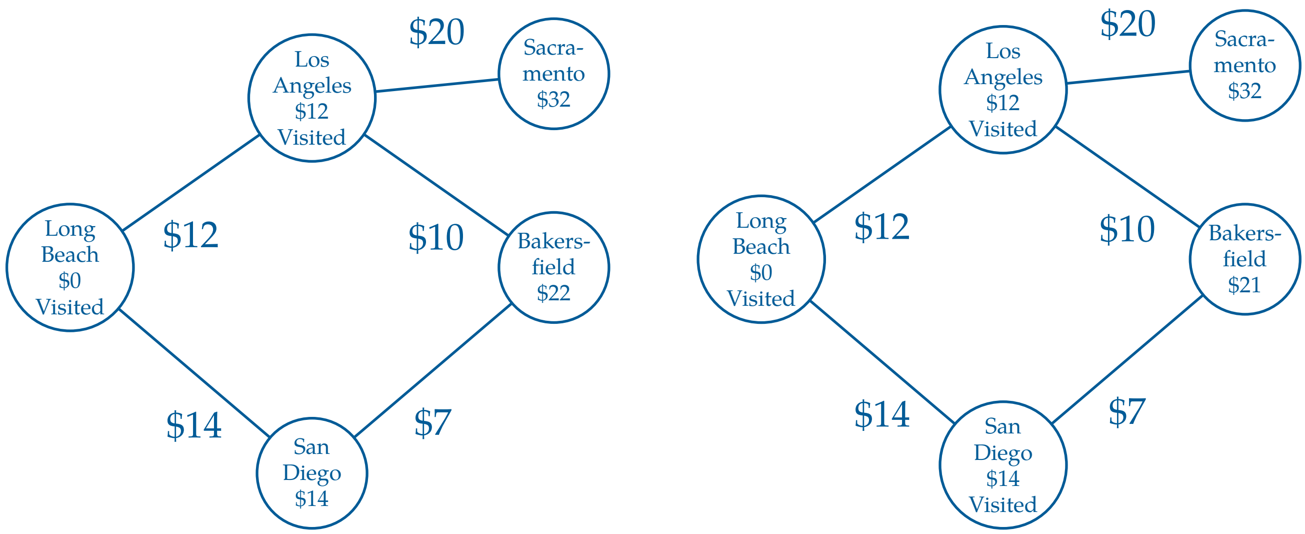
(a) The initialized state of the graph, selecting Long Beach as our origin node.

(b) Marking Long Beach as visited, we change the cost values of Los Angeles and San Diego to their respective edges.

Figure 6.1: A map of weighted cities.

After that, you find the cheapest of the unvisited nodes. In this case, Los Angeles is cheaper than San Diego, so that is the node you will visit next.

You mark all of the unvisited nodes adjacent to Los Angeles, with the price to ship it to Los Angeles plus the cost of shipping the container from Los Angeles to that city (figure 6.2a). Note that Bakersfield is marked with $22.



(a) After visiting Los Angeles.

(b) After visiting San Diego.

Figure 6.2: Visiting the next two minimum cost cities.

Now, the cheapest unvisited node is San Diego, so you mark its neighbors with the cost to ship to San Diego, plus the price to ship from San Diego to the neighbor (figure 6.2b).

Notice that Bakersfield is already labeled with $22 from a route through Los Angeles. But the price would be $21 if you shipped it to Bakersfield via San Diego. Because the new route is cheaper, you change the price to the lower value.

(What does it mean that a node is "visited"? If a node is marked visited, it is marked with a price that won't get any smaller.)

You continue visiting the cheapest unvisited node until all the nodes have been visited. Then you know every node has been marked with its lowest price.

In a big graph, each node may be marked several times in this process — each time with a lower price from a cheaper router.

Of course, once you have the price, you will ask, "What is the route that gets me that price?" So, we will also mark each node with the neighbor from which it would receive the shipment — the previous node. This is easy to do as we execute the algorithm.

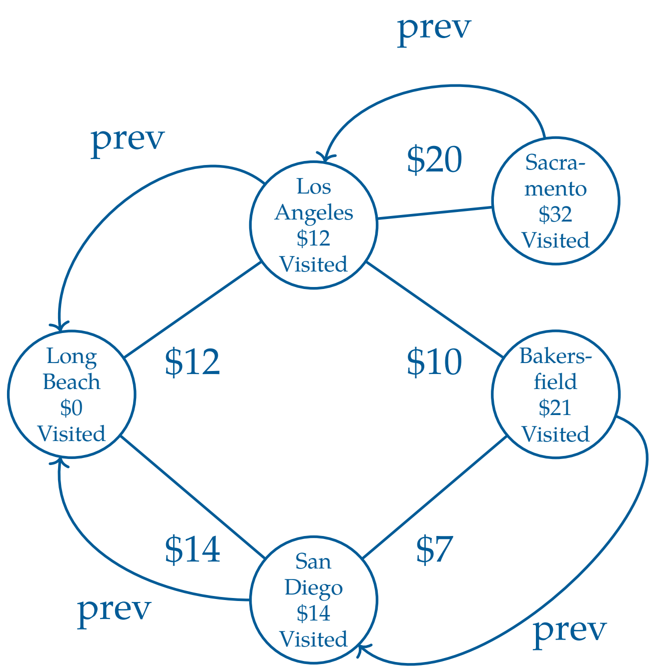


Figure 6.3: Marking each node with a previous pointer.

Now, to figure out the cheapest route from Long Beach to Bakersfield, we start at the destination and follow the `prev` pointer back through San Diego, then to Long Beach.

Exercise 2 Expanding on the Delivery Graph

Notice that the path between Los Angeles to Bakersfield is not used once we add in our previous labels. Why might this be?

Working Space

Answer on Page 41

6.2 Implementation

We do not actually want to sully our graph objects with the three additional pieces of information we need:

- The current minimal cost from the origin node. This is usually called the `dist`, from "distance".
- The neighbor who gives us the current minimal cost. This is usually called `prev`, from "previous".
- Whether the city is visited or now.

So, we will keep them in collections external to the graph.

For example, to keep track of the `dist`, we will have a `dist` dictionary. Each node will be a key, the current minimal cost will be the value. If the node hasn't received even a first cost, we will put in infinity as the cost.

(After the algorithm is run, if the cost of a node is still infinity, that means that it cannot be reached from the origin node.)

We will also have a `prev` dictionary. The final node will be the key, and its previous neighbor will be the value.

Finally, the graph has a list of all the nodes, so we can just keep a set of the unvisited nodes.

Add a method to the `Graph` class that implements Dijkstra's algorithm:

```
    def cost_from_node(self, origin_node):
        # Cost of cheapest path from origin node discovered so far
        # Initially the origin is zero and all the other are infinity
        dist = {k: math.inf for k in self.nodes}
        dist[origin_node] = 0.0

        # The previous city on that cheapest path
        prev = {}

        # All the nodes start as unvisited
        unvisited = set(self.nodes)

        # While there are still unvisited nodes
        while unvisited:

            # Find unvisited node with lowest cost
            min_cost = math.inf
            for u in unvisited:
                if dist[u] < min_cost:
                    current_node = u
                    min_cost = dist[u]

            # If none are less than inf, we are done
            # This happens in graphs that are not connected
            if min_cost == math.inf:
```

```
                return (dist, prev)

            # Remove the lowest cost node from the unvisited list
            unvisited.remove(current_node)

            # Update all the unvisited neighbors
            for edge in current_node.edges:

                # What node is at the other end of this edge?
                v = edge.other_end(current_node)

                # Visited nodes are already minimized, skip them
                if v not in unvisited:
                    continue

                # Is this a shorter route?
                alt = dist[current_node] + edge.cost
                if alt < dist[v]:

                    # Update the distance and prev dicts
                    dist[v] = alt
                    prev[v] = current_node

        return (dist, prev)
```

Append some code to your `cities.py` that tests this method:

```
(cost_from_long_beach, prev) = network.cost_from_node(long_beach)
print(f"\nMinimum costs from Long Beach = {cost_from_long_beach}")
print(f"\nLast city before = {prev}")

nyc_cost = cost_from_long_beach[nyc]

if nyc_cost < math.inf:
    print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")
else:
    print("You can't get to NYC from Long Beach")
```

When you run it, you should get a list of how much it costs to ship a container to each city from Long Beach:

```
Minimum costs from Long Beach = {(node:Long Beach, edges:1): 0.0,
(node:Los Angeles, edges:3): 12.0, (node:Denver, edges:3): 35.0,
(node:Phoenix, edges:3): 31.0, (node:Louisville, edges:3): 49.0,
(node:Cleveland, edges:4): 44.0, (node:Boston, edges:2): 57.0,
(node:New York City, edges:3): 52.0}
```

You will also get a collection of node pairs. What are these? For each node, you get the node that you would pass through on the cheapest route from Long Beach:

```
Last city before = {(node:Los Angeles, edges:3):(node:Long Beach, edges:1),
(node:Denver, edges:3):(node:Los Angeles, edges:3),
(node:Phoenix, edges:3):(node:Los Angeles, edges:3),
(node:Louisville, edges:3):(node:Denver, edges:3),
(node:Cleveland, edges:4):(node:Denver, edges:3),
(node:New York City, edges:3):(node:Cleveland, edges:4),
(node:Boston, edges:2): (node:Cleveland, edges:4)}
```

Your users will not want to read this; give them the shortest path as a list. Add a function to `graph.py` that turns the `prev` table into a path of nodes that lead from the origin to the destination:

```
def shortest_path(prev, destination):

    # Include the destination in the path
    path = [destination]
    current_node = destination

    # Keep stepping backward in the path
    while current_node in prev:

        # What node should come before the current node?
        previous_node = prev[current_node]

        # Insert it at the start of the list
        path.insert(0, previous_node)
        current_node = previous_node

    return path
```

Test that out:

```
if nyc_cost < math.inf:
    print(f"*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")

    path_to_nyc = graph.shortest_path(prev, nyc)
    print(f"*** Cheapest path from Long Beach to NYC: {path_to_nyc ***")}
else:
    print("You can't get to NYC from Long Beach")
```

This should look like this:

```
*** Cheapest path from Long Beach to NYC: [(node:Long Beach, edges:1),
(node:Los Angeles, edges:3), (node:Denver, edges:3), (node:Cleveland, edges:4),
```

```
(node:New York City, edges:3)] ***
```

6.3 Making it faster

On really big networks, doing a full Dijkstra's algorithm would take too long; luckily, there are many methods for getting similar results quickly. When you ask for directions from Google Maps, it does not do a full Dijkstra's Algorithm for every possible route — it would just take too long.

However, there is a way to speed up this implementation. Look at this snippet:

```
            # Find unvisited node with lowest cost
            min_cost = math.inf
            for u in unvisited:
                if dist[u] < min_cost:
                    current_node = u
                    min_cost = dist[u]
```

We are scanning through the list of all unvisited nodes, one by one, looking for the one with the lowest cost. If we kept this list sorted by cost, then the next one to visit would always be the first one in the list. This is done with a *priority queue* – a list that keeps itself sorted by some priority number – in this case the cost. In python, the standard priority queue is `heapq`.

(So why didn't I implement this using `heapq`? For Dijkstra's Algorithm, the nodes' priority – the current cost – changes as we find cheaper routes. `heapq` doesn't handle the changing priority very gracefully.)

In the next chapter, you will make a priority queue class that will work in this case.

Chapter 7

Priority Queues

Chapter 8

Binary Search

As mentioned in the last chapter, you are going to make a priority queue for use with Dijkstra's Algorithm. Using it will look like this algorithm called a *priority queue*:

```
import kpqueue

myqueue = pqueue.PriorityQueue()
myqueue.add(long_beach, 0) # Inserts first city and its cost
myqueue.add(san_diego, 14) # Puts San Diego after Long Beach
myqueue.add(los_angeles,12) # Inserts LA between Long Beach and San Diego
current_city = myqueue.pop() # Returns first city (Long Beach) and removes it
```

Now, if an item gets a new priority, we need to remove it and reinsert it in the new spot.

```
myqueue.add(city_a, 16) # Puts it last in the queue
myqueue.update(city_a, 16, 13) # Moves it to between LA and San Diego
```

8.1 A Naive Implementation of the Priority Queue

Create a file called `kpqueue.py`. Let's do a simple implementation that stores the priority and the data as tuple. And we will keep it sorted by the priority. If two tuples have the same priority, we will sort by the data.

Type this in to `kpqueue.py`:

```
class PriorityQueue:
    def __init__(self):
        self.list = []

    # Return and remove the first item
    def pop(self):
        if len(self.list) > 0:
            return self.list.pop(0)
        else:
            return None

    def __len__(self):
        return len(self.list)
```

```

    def update(self, value, old_priority, new_priority):
        old_pair = (old_priority, value)
        self.list.remove(old_pair)
        self.add(value, new_priority)

    def add(self, value, priority):
        pair = (priority, value)
        # Add it at the end
        self.list.append(pair)
        # Resort the list
        self.list.sort()
```

This will work fine, but it could be much more efficient:

- Every time we add a single element, we resort the whole list.
- The function `remove` is searching the list sequentially for the item to delete.

In a minute, we will revisit these inefficiencies and make them better.

8.2 Using the Priority Queue

We are going to change `graph.py` to use the priority queue. While we are doing so, why don't we also shrink the memory footprint of our program a bit.

Notice that as the algorithm is running, each node is in one of three states:

- Unseen: In the earlier implementation, these were the nodes with `math.inf` as their cost.
- Seen, but not finalized: These are "unvisited", but don't have `math.inf` as their cost.
- Finalized: These are the "visited" nodes — we know that their cost won't decrease any more.

We can shrink the memory foot print by not putting the unseen into the dist dictionary at all. And instead of a separate set for "unvisited", what if we moved finalized nodes and their distances into a separate dictionary?

Rewrite the `cost_from_node` function in `graph.py`:

```
            # Visited nodes are already minimized, skip them
            if v in finalized_dist:
                continue
```

```

                # What is the cost to this neighbor?
                alt = current_node_cost + edge.cost

                # Is this the first time I am seeing the node?
                if v not in seen_dist:

                    # Insert into the seen_dict, prev, and priority queue
                    seen_dist[v] = alt
                    prev[v] = current_node
                    pqueue.add(v, alt)

                else: # v has been seen. Is this a cheaper route?
                    old_dist = seen_dist[v]
                    if alt < old_dist:
                        # Update the seen_dict, prev, and priority queue
                        seen_dist[v] = alt
                        prev[v] = current_node
                        pqueue.update(v, old_dist, alt)

        return (finalized_dist, prev)
```

This should be have exactly the same result, except for the unreachable nodes. If you have a graph that is not connected, there will be nodes that cannot be reached from the origin. In the old version, these had a cost of `math.inf`. Now, they will just not be in the dictionary at all. So, change `cities.py` to deal with this:

```
if nyc in cost_from_long_beach:
    nyc_cost = cost_from_long_beach[nyc]
    print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")

    path_to_nyc = graph.shortest_path(prev, nyc)
    print(f"\n*** Cheapest path from Long Beach to NYC: {path_to_nyc} ***")
else:
    print("You can't get to NYC from Long Beach")
```

If you run `cities.py` now, it should behave exactly like the old version.

However, there is a bug. It will rear its head if two cities with the same cost are in the priority queue together. Change `cities.py` so that Denver and Phoenix have the same cost:

```
graph.WeightedEdge(12, long_beach, los_angeles)
graph.WeightedEdge(19, los_angeles, denver)
graph.WeightedEdge(19, los_angeles, phoenix)
```

Now try running it. You should get an error:

```
TypeError: '<' not supported between instances of 'Node' and 'Node'
```

What happened? The `loc_for_pair` method is comparing tuples made up of a `float` and a `Node`. The `float` comes first in the tuple, so that is compared first. However, if the two tuples have the same priority, it then compares nodes.

The error statement says, “Nodes don’t have a less-than method; I don’t know how to compare them.”

Each `Node` lives at an address in memory. You can get that address as a number using the `id` function. The ID is unique and constant over the life of the object. It is a rather arbitrary ordering, but it will work for this problem. Add a method to your `Node` class:

```
    # Nodes will be ordered by their location in memory
    def __lt__(self, other):
        return id(self) < id(other)
```

Fixed.

Now, let’s make the priority queue more efficient.

8.3 Binary Search

The phone company in every town used to print a thing called a phone book. The names and phone numbers were arranged alphabetically. As you might imagine, these books often had more than a thousand pages.

If you were looking for “John Jeffers”, you would not start at the first page and read sequentially until you reached his name. You would open the book in the middle, and see a name like “Mac Miller”, then think “Jeffers comes before Miller”. You would then split the pages in your left hand in half and see a name like “Hester Hamburg” and think “Jeffers comes after Hamburg”. You would then split the pages in your right hand, and continue on like this until you found the page with “John Jeffers” on it. That is binary search.

Binary Search is a search algorithm that finds the position of a target value within a sorted array. The binary search algorithm works by repeatedly dividing the search interval in half. If the target value is equal to the middle element of the array, the position is returned. If the target value is less than or greater than the middle element, the search continues in the lower or upper half of the array, respectively.

8.4 Algorithm

The binary search algorithm can be described as follows:

1. If the array is empty, the search is unsuccessful, so return ”Not Found”.
2. Otherwise, compare the target value to the middle element of the array.
3. If the target value matches the middle element, return the middle index.
4. If the target value is less than the middle element, repeat the search with the lower half of the array.
5. If the target value is greater than the middle element, repeat the search with the upper half of the array.
6. Repeat steps 2-5 until the target value is found or the array is exhausted.

```
class PriorityQueue:
    def __init__(self):
        self.list = []

    # Return and remove the first item
    def pop(self):
        if len(self.list) > 0:
            return self.list.pop(0)
        else:
            return None

    def __len__(self):
        return len(self.list)

    def add(self, value, priority):
        pair = (priority, value)
        i = self.loc_for_pair(pair)
        self.list.insert(i, pair)

    def update(self, value, old_priority, new_priority):
        old_pair = (old_priority, value)
        i = self.loc_for_pair(old_pair)
        del self.list[i]
        self.add(value, new_priority)

    def loc_for_pair(self, pair):
        # The range where it could be is [lower, upper)
        # Start with the whole list
        lower = 0
        upper = len(self.list)

        while upper > lower:
            next_split = (upper + lower) // 2
            v = self.list[next_split]
```

```
            if pair < v:  # pair is to the left
                upper = next_split
            elif pair > v:  # pair is to the right
                lower = next_split + 1
            else: # Found pair!
                return next_split
        return lower
```

If you try running it now, it should work perfectly.

You now have a graph class that will find the cheapest path quickly, even if it has thousands of nodes with thousands of edges.

Chapter 9

Other Graph Algorithms and Proofs

Now that you are familiar with Dijkstra's algorithm for finding the shortest path in a graph, you are well-equipped to understand more graph algorithms. This document will discuss two other important algorithms: *Depth-First Search* (*DFS*) and the Bellman-Ford algorithm.

9.1 Depth-First Search

Depth-First Search (DFS) is an algorithm for traversing or searching tree or graph data structures. DFS uses a stack (or sometimes recursion, which uses the system stack implicitly) to explore the graph in a depthward motion until it hits a node with no unvisited adjacent nodes, then it backtracks.

The procedure is as follows:

1. Push the root node into the stack.
2. Pop a node from the stack, and mark it as visited.
3. Push all unvisited adjacent nodes into the stack.
4. Repeat steps 2 and 3 until the stack is empty.

DFS is particularly useful for solving problems like connected-component detection in graphs and maze-solving.

9.2 Bellman-Ford Algorithm

The Bellman-Ford Algorithm is another shortest path algorithm like Dijkstra's. However, unlike Dijkstra's Algorithm, Bellman-Ford can handle graphs with negative weight edges.

The algorithm works as follows:

1. Assign a tentative distance value for every node: set it to zero for our initial node

and to infinity for all other nodes.

2. For each edge (u, v) with weight w, if the current distance to v is greater than the distance to u plus w, update the distance to v to be the distance to u plus w.
3. Repeat the previous step $|V| - 1$ times, where $|V|$ is the number of vertices in the graph.
4. After the above steps, if you can still find a shorter path, there exists a negative cycle.

If the graph does not contain a negative cycle reachable from the source, the shortest paths are well-defined, and Bellman-Ford will correctly calculate them. If a negative cycle is reachable, no solution exists, but Bellman-Ford will detect it.

9.3 Prim's Algorithm

Prim's Algorithm is minimum spanning tree FIXME https://github.com/KontinuaFoundation/sequence/issues/315

9.4 Breadth-First Search

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 6)

```
SELECT bike_id FROM bike WHERE purchase_price > 330 AND brand='Trek'
```

Answer to Exercise 2 (on page 25)

Los Angeles $\rightarrow$ Bakersfield is not chosen because it does not give the best known cost to Bakersfield. The `prev` pointer for Bakersfield goes to San Diego instead, since \$21 $<$ \$22.

The tree that is generated is called a *predecessor tree* or *explored tree* of the resulting edges formed by only the previous edges. We can use this to track shortest paths from a given origin node.

Index